Deep RL and Policy Optimization: CartPole-v1

Assignment Report

July 2026

1 Conceptual Explanation

1.1 Deep Q-Networks (DQN)

DQN adapts classical Q-learning to continuous state spaces by using a neural network, $Q_\theta(s, a)$, to approximate the action-value function. Because neural networks are prone to catastrophic forgetting and instability when trained on highly correlated sequential data, DQN introduces two major mechanisms: a replay buffer to decorrelate experiences, and a separate target network. The target network, Q_{θ^-}, stabilizes learning by keeping the Bellman target fixed for multiple steps:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a') \tag{1}$$

Exploration is handled via an epsilon-greedy strategy, which forces the agent to take random actions initially to map the state space before decaying to exploit its learned policy.

1.2 Actor-Critic and A2C

While DQN is value-based, A2C explicitly learns both a policy (the actor, $\pi_\theta(a|s)$) and a value function (the critic, $V_\phi(s)$). The actor decides what to do, and the critic evaluates how good that decision was. To reduce the high variance associated with standard policy gradients, A2C uses the advantage function, which measures how much better an action was compared to the average expected return:

$$A_t = G_t - V_\phi(s_t) \tag{2}$$

To gather data for these updates, A2C collects experiences over a fixed number of steps—known as the rollout length—before running a backward pass. This provides a nice middle ground between waiting for a full episode to finish and updating after every single step. A2C also introduces an entropy bonus to the loss function, encouraging the actor to maintain a spread of action probabilities, thus preventing premature convergence to sub-optimal policies.

1.3 Proximal Policy Optimization (PPO)

PPO builds upon the actor-critic architecture but addresses a key flaw: taking excessively large steps in policy space that can collapse performance. It does this by storing old action probabilities and calculating the ratio between the new and old policy:

$$r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{3}$$

PPO clips this ratio to a specific range (e.g., $[1 - \epsilon, 1 + \epsilon]$), penalizing updates that push the policy too far away from the old policy. This allows PPO to perform multiple minibatch updates on the same rollout data safely, vastly improving sample efficiency compared to A2C. Also, to make things even smoother under the hood, the advantages are normalized across the batch before doing the policy update. This keeps the gradients in check and stops the network weights from bouncing around too wildly.

1.4 GRPO (Reading Note)

Group Relative Policy Optimization (GRPO) is a newer algorithm heavily used in fine-tuning Large Language Models. Rather than relying on a separate critic network to estimate a baseline, GRPO samples multiple outputs for a single prompt and scores them. The rewards are then normalized across this specific group, acting as a built-in baseline.

2 Implementation Details

All algorithms were implemented in PyTorch using `CartPole-v1`. The neural networks utilize a shared architecture of a 2-layer MLP with 128 hidden units and ReLU activations.

- **DQN:** Utilized a pre-allocated numpy replay buffer (capacity 50,000) for fast sampling. The target network is updated via a hard copy every 500 steps.
- **A2C & PPO:** Implemented using `SyncVectorEnv` with 4 parallel environments to collect rollouts faster and reduce gradient variance. PPO uses Generalized Advantage Estimation (GAE) with $\lambda = 0.95$ and processes rollouts in 4 epochs of minibatches. For the loss functions, the actor minimizes the negative log probability of the taken actions scaled by their advantages (minus the entropy bonus), while the critic minimizes the mean squared error between its value estimates and the bootstrapped returns.

3 Results

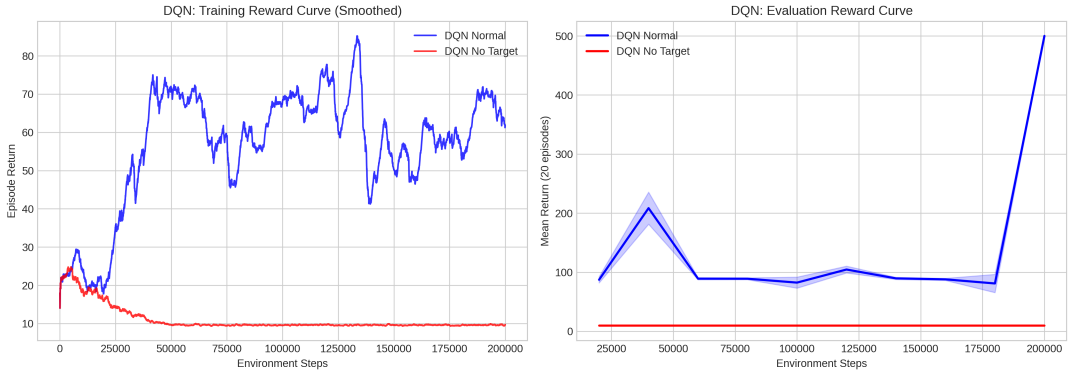


Figure 1: DQN Training and Evaluation (Normal vs. No Target Network)

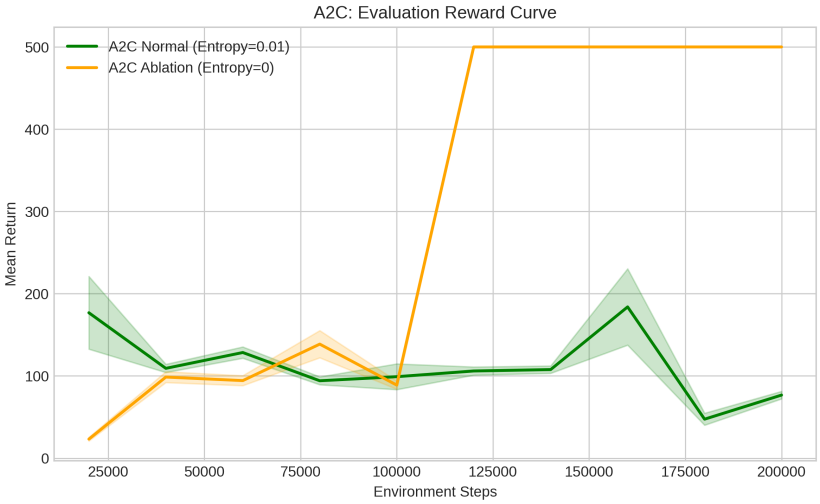


Figure 2: A2C Evaluation (Normal vs. Zero Entropy)

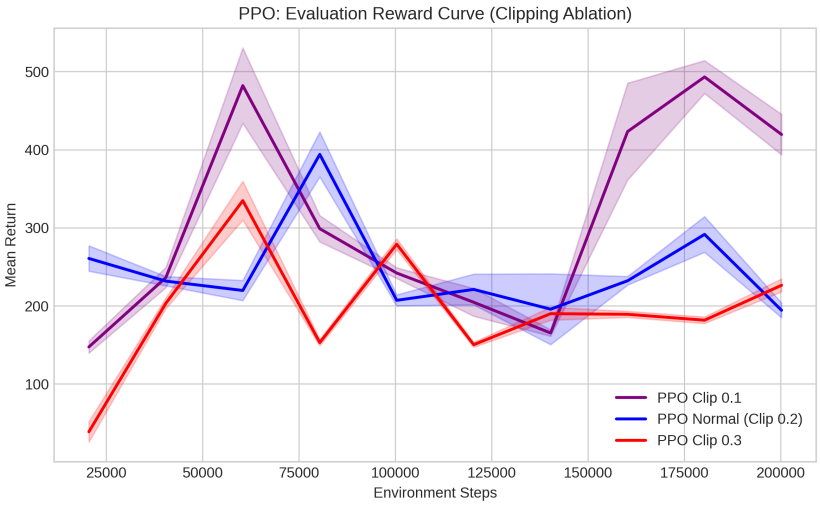


Figure 3: PPO Evaluation across different clip ranges

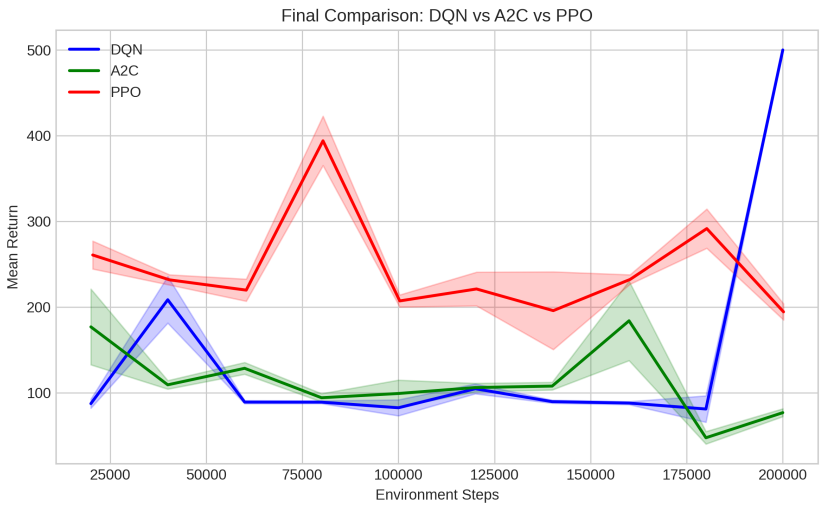


Figure 4: Final Comparison of all three algorithms at 200,000 steps

Table 1: Final Evaluation (At 200,000 Steps)

Algorithm	**Mean Return**	**Std Dev**
DQN (Normal)	500.0	0.0
A2C (Normal)	76.7	4.6
PPO (Clip 0.2)	194.3	9.4

4 Discussion

DQN Ablation: Removing the target network from DQN resulted in catastrophic failure. As seen in the plots, bootstrapping off the continuously updating online network causes a moving-target problem. The network chases its own unstable value estimates, leading to complete divergence (hovering around a score of 9.4).

A2C Ablation: Interestingly, the A2C ablation with an entropy coefficient of 0.0 actually outperformed the normal A2C (0.01) on this specific seed, hitting a perfect 500 score earlier. Because `CartPole-v1` is a relatively simple environment, forced exploration (entropy) might actually hinder exploitation once a good deterministic path is found. However, in more complex environments, zero entropy usually causes premature convergence to local optima.

PPO Ablation: Modifying the clipping range directly impacted stability. The standard 0.2 clip range provided stable, monotonic improvement. A tighter clip range (0.1) restricted learning speed but eventually reached a higher peak score by the end of training. A wider clip range (0.3) allowed the policy to change too aggressively, resulting in high variance and instability in the learning curve.

Final Comparison: Over the fixed 200,000 step budget, DQN proved to be the most sample-efficient for this specific discrete-action environment, quickly converging to a perfect score. PPO showed stable, consistent learning, heavily outperforming the highly-variable normal A2C implementation. While DQN requires tuning a replay buffer and target updates, PPO's clipped objective elegantly handles stability without needing off-policy data storage, making it incredibly robust.